```
    s2 = nw.addSwitch( 's2' )
    s3 = nw.addSwitch( 's3' )
    info( 'Create links\n' )
    nw.addLink( h1, s1 )
    nw.addLink( h2, s1 )
    nw.addLink( h3, s2 )
    nw.addLink( h4, s2 )
    nw.addLink( h5, s3 )
    nw.addLink( h6, s3 )
    info( 'Start network\n')
    nw.start()
    info( 'Run CLI\n' )
    CLI(nw)
    info( 'Stop network' )
    nw.stop()
if __name__ == '__main__':
    setLogLevel( 'info' )
    mytopo()
```

In the above code, the h1 and h2 hosts are connected to switch s1; h3 and h4 hosts are connected to switch s2; and h5 and h6 hosts are connected to switch s3. All the hosts are connected to controller c0 through the above mentioned virtual switches. In the above code, the following class and methods are included.

Mininet: Class to create and manage a network topology.

addHost(): Adds a host to a topology.

addSwitch(): Adds a switch to a topology.

addLink(): Adds a bi-directional link between two nodes in the topology.

Simulation of the created topology in Mininet

To simulate the above custom topology, run the *mytopo.py* file on the terminal using the following command:

```
# sudo python mytopo.py
```

When we run the above command, it configures the components of topology and starts the command line interface (CLI). To test the working and connections set in the topology, use the following command:

```
mininet> pingall
```

Use the *dump* command to see the information about all the nodes of the created topology. Figure 5 shows the connectivity between all the hosts of this topology.

```
malaram@malaram-VPCEG25EN: ~
File  Edit  View  Search  Terminal  Help
malaram@malaram-VPCEG25EN:~$ sudo python mytopo.py
Adding controller
Adding hosts
Adding switch
Creating links
Starting network
*** Configuring hosts
h1 h2 h3 h4 h5 h6
*** Starting controller
c0
*** Starting 3 switches
s1 s2 s3 ...
Running CLI
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 X X X
h2 -> h1 X X X
h3 -> X X h4 X
h4 -> X X h3 X
h5 -> X X X h6
h6 -> X X X h5
*** Results: 80% dropped (6/30 received)
mininet> dump
<Host h1: h1-eth0:10.0.0.11 pid=11524>
<Host h2: h2-eth0:10.0.0.12 pid=11526>
<Host h3: h3-eth0:10.0.0.13 pid=11528>
<Host h4: h4-eth0:10.0.0.14 pid=11530>
<Host h5: h5-eth0:10.0.0.15 pid=11532>
<Host h6: h6-eth0:10.0.0.16 pid=11534>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None pid=11539>
<OVSSwitch s2: lo:127.0.0.1,s2-eth1:None,s2-eth2:None pid=11542>
<OVSSwitch s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None pid=11545>
<Controller c0: 127.0.0.1:6653 pid=11517>
mininet>
```

Figure 5: Simulation of custom topology (mytopo.py)

Tracing the network using Wireshark

Wireshark is a network protocol analyser that monitors the network and examines OpenFlow packets that are

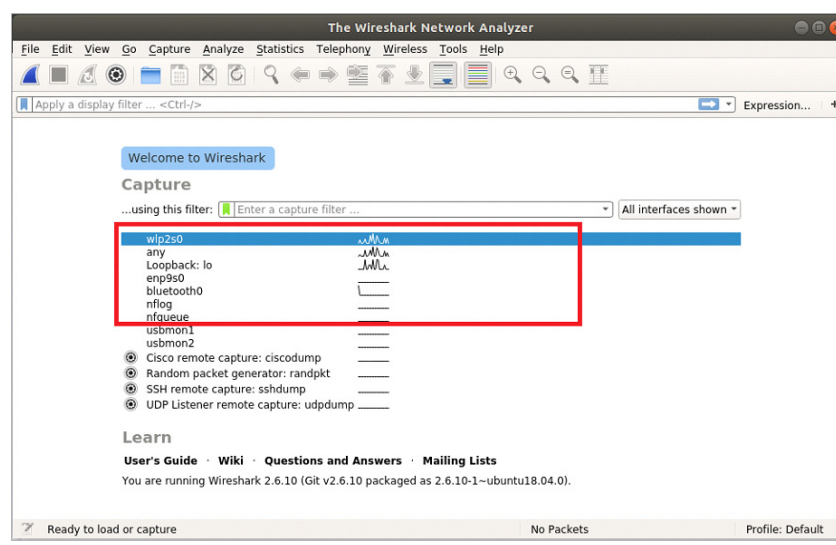

Figure 6: Wireshark window